Content-preserving Text Watermarking through Unicode Homoglyph Substitution

Stefano Giovanni Rizzo, Flavio Bertini, Danilo Montesi
Department of Computer Science and Engineering
University of Bologna, Italy
stefano.rizzo8@unibo.it, flavio.bertini2@unibo.it, danilo.montesi@unibo.it

ABSTRACT

Digital watermarking has become crucially important in authentication and copyright protection of the digital contents, since more and more data are daily generated and shared online through digital archives, blogs and social networks. Out of all, text watermarking is a more difficult task in comparison to other media watermarking. Text cannot be always converted into image, it accounts for a far smaller amount of data (eg. social network posts) and the changes in short texts would strongly affect the meaning or the overall visual form. In this paper we propose a text watermarking technique based on homoglyph characters substitution for latin symbols[1]. The proposed method is able to efficiently embed a password based watermark in short texts by strictly preserving the content. In particular, it uses alternative Unicode symbols to ensure visual indistinguishability and length preservation, namely content-preservation. To evaluate our method, we use a real dataset of 1.8 million New York articles. The results show the effectiveness of our approach providing an average length of 101 characters needed to embed a 64bit password based watermark.

Categories and Subject Descriptors

I.7.m [**Document and Text Processing**]: Miscellaneous; K.5.m [**Legal Aspect of Computing**]: Miscellaneous; K.6.5 [**Management of Computing and Information Systems**]: Security and Protection

Keywords

Text Watermark, Unicode Characters, Copyright Protection, Document Authentication, Tampering Detection

1. INTRODUCTION

In the last decades, one of the noticeable behaviour is the massive production of digital contents, like texts, data, images and videos. These documents, namely digital contents, meet several kind of users' needs both in professional activities and social interactions. Moreover, the proliferation of social media and mobile devices offer users many platforms for sharing digital contents. On the flip side, many of them are copyrighted and it is important to protect the intellectual property rights of the authors. The problem is more meaningful since digital contents are frequently used as evidence in lawsuit. Nevertheless, as highlighted in [16], it is quite complex to reach a good balance among intellectual property rights, freedom of expression, information, privacy and free choice. The critical issue behind this complex balance is that the classic digital rights protection techniques prevent illegal use of the copyrighted digital contents, but sometimes they impose restrictions within the confines of a permitted use.

There are several illicit actions concerning these digital contents, as illegal copying, tampering, forgery, authentication and theft. As a result, different techniques are developed and can be categorized in three different classes: cryptography, steganography and watermarking [12]. While cryptography algorithms make unreadable the information by applying a kind of permutation or substitution to the original content, the steganography algorithms provide techniques to hide new information into the carrier, that is a readable document. Whereas watermarking algorithms ensure the authentication and the copyright protection by applying a watermark to the digital content.

Intuitively, all these methods can be used with any kind of digital contents, however, watermarking is the most balanced technique for sharing not obfuscated information combined with preserving copyright [25]. In particular, watermarking methods can be applied in various contexts, like establishing the authorship of a digital content, monitoring the broadcasting process, identifying unauthorized users and distrusting a tampered digital content. Also, watermarking can be adopted, up to an acceptable distortion, to protect dynamically generated contents from databases [7].

According to literature [13], watermarking can be categorized as following:

- *Invisible or Visible* - The watermark is invisible if it is hidden in the carrier, that is the digital content and it does not appear to the user. Otherwise, a visible watermark is noticeable to the user.
- *Readable or Detectable* - The watermark is readable if the user can clearly read it. Otherwise, a detectable watermark needs a detection function to check if it exists or not.

[1]A prototype that implements our method can be tested here: http://smartdata.cs.unibo.it/watermark/

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from Permissions@acm.org.
IDEAS '16, July 11-13, 2016, Montreal, QC, Canada
Copyright 2016 ACM 978-1-4503-4118-9/16/07 $15.00.
http://dx.doi.org/10.1145/2938503.2938510.

- *Robust, Fragile, Semi-Fragile* - A robust watermark is detectable and not erasable, and it is most suitable for copyright protection. A fragile watermark is detectable, but it can be altered and erased, it is used for integrity authentication. Then, a semi-fragile watermark is suited for content authentication.
- *Blind or Non Blind* - A watermark belongs to the blind category if while extracting process there is no need of the original digital content, otherwise it belongs to non blind category.
- *Zero Watermarking* - The digital content is not modified by the watermarking process, but the algorithm use its characteristics to generate the watermark.
- *Simple or Multiple* - A simple watermark is applied only once. Otherwise a multiple watermark is inserted more times without affecting the previous watermarking processes.

In [20], the researchers identify the features usually required to a watermarking method. *Robustness* is the ability to resist to attacks and processing operations. *Security* is the ability to not be removed or altered without having full awareness of the method or the watermark. *Verifiability* is the ability to irrefutably prove the ownership of the digital content. *Cost* represents the computational cost required both in embedding and extraction process. Finally, *data payload* is maximum amount of information (number of bits) that can be embedded in the original digital content.
Our work focuses on text watermarking among all digital content watermarking techniques. An immense amount of textual knowledge increases daily and represents one of the largest bunch of digital contents that people can explore online. Furthermore, short text messages are used for government, commerce and mobile banking communications, as well as social media and micro blogging posts. For these reasons, authorship protection of textual documents, including short text message, cannot be neglected any more. Nevertheless, text watermarking, the domain of our work, is the most challenging task in comparison to other digital contents watermarking. In particular, a text watermarking algorithm must work with some additional constraints, like a low embedding bandwidth, in term of message length; a restricted set of transformations that are imposed to ensure original text readability and a restricted number of alternative syntactic and semantic permutations.
Text watermarking algorithms can be classified in:

- *Image-based Techniques* - In this approach the text is transformed in an image in order to embed the watermark. Obviously, these methods are language independent and preserve the length of the text, however they modify the nature of the original document.
- *Syntactic Techniques* - Assuming that sentences have different language depending structures, these methods transform these text structures to hide the watermark.
- *Semantic Techniques* - These methods are based on the text contents to embed the watermark. In particular, they use verbs, nouns, prepositions, but even spelling and grammar rules to permute the contents and insert the watermark.
- *Structural Techniques* - It is the most recent approach. The text is not altered, rather it is used to embed the watermark. In particular, these methods exploit double letters occurrences, lines-shift or words-shift encoding and Unicode standard.

In this paper, we present a new text watermarking method able to embed a password based watermark without altering the content and preserving the length, namely having *content-preserving* features. In particular, the proposed structural watermarking technique belongs to *invisible*, *detectable* and *blind* classes.
Our method generates the watermark by applying a hash function that combines the structural characteristic of the text and a user password. Then, the watermark is embedded in the text by exploiting homoglyph characters, that is our main contribution. A similar technique has been recently used to hide a message in text [9]. Homoglyph characters are characters (symbols, numbers and letters) that look very similar on the screen and in print, but whose coding is completely different. In practice, according to the watermark bits sequence, we replace the original characters of the text with an indistinguishable subset of latin homoglyph characters. The Unicode confusable characters, namely the homoglyph characters, are listed by Unicode consortium and look confusingly similar from each others [6].
The proposed method has two significant features, it leaves visible-unaltered the text and preserves the length of the original text. The first one strongly depends by the font used, however, homoglyph characters allow to cover the most used font families, as shown in the evaluation section. The second feature is quite complex to ensure in short texts. The combination of these two traits allows to use our method in several new contexts, as: short message communications, microblogging platforms and social networks.
The rest of the paper is organized as follows. In Section 2, we review literature works related to text watermarking methods. In Section 3, we describe our text watermarking methodologies, including watermark generation, embedding and extraction and authorship verification. The evaluations of our method is discussed in Section 4. Some concluding remarks are made in Section 5.

2. RELATED WORKS

In this section, we consider only password based text watermarking methods (blind and non-blind), where the carrier text is modified to embed a watermark and authorship verification is possible. Text watermarking approaches in literature have been classified in three main categories [11, 13]: image-based, syntactic and semantic. All these approaches can be applied to limited scenarios or strongly alter the content of text. After highlighting the core ideas behind these approaches, more recent a fourth class is identifies, that is the structural category. Unicode-based methods belong to this last class.

2.1 Image-based methods

The image-based watermark is the most known approach to text watermark and the first one to be investigated. In this

approach the text is first printed as an image, and then a watermark is applied on this image, hiding a payload (usually a keyed hash) in the pixels of the image. In greyscale document images, watermark data is embedded by modulating the luminance of pixels [4] or by edge direction histograms to carry the watermark signal [15]. Changes and irregularities in text images can be more perceptible than in pictures [8], especially when there is a clean, black and white separation between the foreground text and the background. For this reason most of the image-based works leverage the peculiarities of text images, encoding a watermark by altering text elements at levels higher than the atomic pixels: blocks of text, lines, words, characters, strokes. An effective method is the horizontal and vertical shifting of text units, such as shifting the words left or right and the lines (or blocks) up and down [5, 17]. The position of these units are slightly moved accordingly to the watermark data. A similar method is to alter the inter-word spaces to encode the watermark pattern [10, 14]. Another method is based on alteration of the characters [5], for instance by prolonging strokes or serifs of letters [1]. We consider the image-based approach more as a workaround to text watermarking than as a solution. Image-based watermarking reduces the problem to the more researched scenario of image watermarking, actually binding its application to limited contexts (i.e. photocopying printed paper). In many actual scenario, such as microblogging, texting or emails, it would be unnatural and impractical to post the screenshot of a text instead of the text itself.

2.2 Syntactic methods

In the syntactic approaches, the syntactic structure of natural language text is transformed to embed a watermark. After the syntactic tree is built, the sentences' structure are transformed to encode the watermark bits, with operations such as clefting or passivization [2]. Other meaning-preserving morpho-syntactic transformation can be also applied [18]. For instance, the linguistic notion of possession can be expressed either with the preposition 'of' or with the suffix '-s'. While the syntactic approaches ideally preserve the semantic interpretation of sentences, the overall text content is strongly altered and may not reflect the author's original message (i.e. in poetry, songs or quotes). Moreover more than one sentence is likely needed to embed the whole watermark, thus this approach may be not suitable in limited context (e.g. Twitter posts, SMS texts).

2.3 Semantic methods

The semantic watermark approaches, as the syntactic approaches, make use of Natural Language Processing to encode the watermark. These methods leverage the similitude of terms to substitute words accordingly to the watermark [24]. Sometimes the semantic approach is mixed with the syntactic approach [23] and a bigger transformation space is obtained, that is more space available for watermark embedding. Like the syntactic approaches, these methods make extensive modifications to the original text, producing a visibly different document and altering the author's content.

2.4 Structural methods

Given the increased diffusion of the Unicode standard in information systems, data hiding through Unicode transformation is receiving more interest lately. In Unicode approaches for information hiding and text watermarking, the payload data is embedded using invisible Unicode symbols. The Unicode standard provides many different white spaces. These whitespace symbols have been used to encode long payloads in Microsoft Word documents by replacing the whitespaces in text as well as the empty line with patterns of whitespaces [21]. Other than whitespaces, some Unicode symbols with specific purposes are totally invisible and do not fill spaces in text. These symbols, together with whitespaces, have been used to embed watermark in HTML content [19]. Unlike the syntactic and semantic approaches, these last works preserve the text content. However the application contexts are still restricted and the text length is not preserved, since an overhead of symbols are added in the embedding process.

3. OUR APPROACH

The proposed method for text watermarking preserves both the appearance and the content without converting the text into image. These features make the approach suitable for social networks, forums and text chat. Following the categorization used so far, our approach falls under this classification:

- Invisible (totally or partially): replacing some characters and spaces with visually similar symbols, the watermark can be totally or partially invisible. The visibility strictly depends on the text font used.
- Detectable, fragile and blind: the watermark can be detected and extracted using the symbols mapping. Nonetheless the watermark is verifiable: only the owner of the password (secret key) can prove his authorship.
- Content and length preserving: in order to embed the watermark without altering the content, we alter the underneath unicode representation. In particular, the latin symbols are switched between the original unicode representations and other similar symbols with a different coding.

Compared to other text watermarking methods, our method is more efficient and effective, sharing different strong points of syntactic, semantic and structural methods. The efficiency is due to the symbol-to-symbol mapping compared to the more complex NLP semantic techniques [2]. It is not dependent on particular file formats [21] or markup languages [19] so it can be applied to any unicode text. Finally it is not vulnerable to selection of text for *copy&paste*, conversely to other structural methods [21, 19] in which the watermark is appended at the end of lines or markup elements. The main point of failure of the proposed method is the rewriting of the text, manual or through OCR tools (Optical Character Recognition). This is an intrisic drawbacks for all those text watermarking methods that do not alter the text content, such as image-based and structural methods.

3.1 Unicode confusables

Among the 120 thousand of Unicode symbols it occurs that some symbols are totally or partially indistinguishable from

	Bit 0	Bit 1
Symbol	Original code	Duplicate code
-	0x002d	0x2010
;	0x003b	0x037e
C	0x0043	0x216d
D	0x0044	0x216e
K	0x004b	0x212a
L	0x004c	0x216c
M	0x004d	0x216f
V	0x0056	0x2164
X	0x0058	0x2169
c	0x0063	0x217d
d	0x0064	0x217e
i	0x0069	0x2170
j	0x006a	0x0458
l	0x006c	0x217c
v	0x0076	0x2174
x	0x0078	0x2179

Table 1: Subset of confusable symbols used to encode the watermark. Original and duplicate code is shown for each symbol.

others, though with an underneath different numerical representation. The similarity between different symbols in Unicode represents a security problem well known to the Unicode Consortium [6]. For this reason, the Unicode Consortium publishes and updates a list[2] of "confusable" symbols, to help determining when two different strings are visually confusable.

These duplicates can be exploited to encode the bits of an hidden payload, such as the watermark password-based hash, in the original text. For instance, if each symbol of the latin alphabet had an indistinguishable duplicate in the unicode table (i.e. two codes for the same symbol), we could use the first representation for the bit "0" of the hash and the duplicate for the bit "1". In this example we would need 64 characters to encode a 64bit hash. The actual scenario is slightly different, since only a subset of the latin letters have one or more visually similar symbols in Unicode. Starting from the whole list of confusable symbols, we took the most visually similar symbols for latin letters and white space. In particular, we select a set of 16 letters and punctuations with almost or totally indistinguishable symbols, Table 1, and a set of 8 white space similar symbols, Table 2.

In order to find out the most similar symbols, we tested their homoglyphs under the most used font families in modern desktop and web applications, obtaining imperceptible differences in most used sans-serif fonts. The reader can find the evaluation results in Section 4.

3.2 Watermark generation with password

In order to produce and verify the watermark a cryptographic hash function is used in combination with a password. A cryptographic hash function in the watermarking context is a one-way function that takes a text t, a password k and produces in output a value of fixed size (hash).

In our choice of a cryptographic hash function, two properties play an important role: the security robustness of the function and the size of the produced hash. In order to be capable of watermarking short texts we need a small payload, so a smaller hash is preferred, however this can negatively affects its robustness. SipHash [3] is a MAC (Message Authentication Code) function that works on variable-length message and 128bit secret keys. It is designed to be highly secure even for short messages despite being a short sequence of 64 bits. Moreover, SipHash is claimed to be much faster on short input than other strong MACs. These features make SipHash the ideal MAC function in our context, allowing us to successfully embed a secure watermark on very short text using Unicode. Nonetheless our approach does not depend on a specific hash function, therefore in Section 4 we evaluate the embedding capacity of 3 more hash functions of different sizes.

[2] http://www.unicode.org/Public/security/8.0.0/confusables.txt

White space	Bits	Unicode
Space	000	0x0020
En Quad	001	0x2000
Three-per-em Space	010	0x2004
Four-per-em Space	011	0x2005
Punctuation Space	100	0x2008
Thin Space	101	0x2009
Narrow No-break Space	110	0x202f
Medium Mathematical Space	111	0x205f

Table 2: Encoding for white space symbols. In order to encode 3 bits in a single white space, 8 different white spaces are used in total.

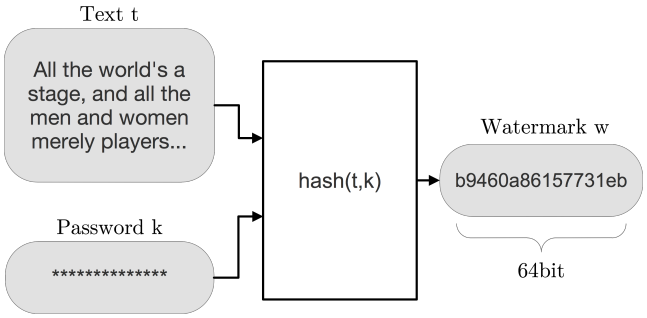


Figure 1: Given the original text t and a password k, the SipHash function generates a cryptographic hash, representing the watermark w to be embedded.

In Figure 1 is shown how the SipHash cryptographic hash function can be applied to a sample text t using a password k to produce a 64bit hash, representing the text watermark. Since only the author with his key is able to recompute the hash, this watermarking allows a secure authorship verification.

3.3 Unicode watermark embedding

Once a watermark is generated using an hash function, this is embedded in the original text. The embedding algorithm, together with the Unicode confusable mapping in Table 1 and 2, represent the main contribution of this work to text watermarking. Switching between original symbols and identical, homoglyph symbols, the algorithm makes possible to embed a payload without altering the readable content and the length of the original text.

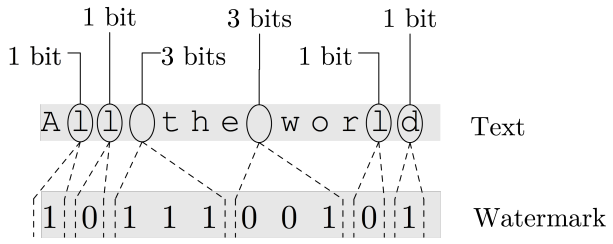


Figure 2: Only the symbols with a related *duplicate* in Tables 1 and 2 are used to embed the watermark.

The algorithm scans each character in the original text looking for a confusable symbol. In Figure 2 is shown how the algorithm detect and map confusable symbols to watermark bits. When a confusable symbol is found, this can be used to encode one or three bits in its Unicode representation: one bit if the confusable is a symbol in Table 1, three bits if the confusable is a space character in Table 2.
When the confusable is a non-space symbol, the most significant bit is removed from the watermark binary array. If this bit is 0 than the original symbol is used, otherwise the duplicate symbol in Table 1 is used instead.
When the confusable is a white space, the three most significant bits are removed from the watermark array. If the three bits are equal to 000 than the original white space is left, otherwise it is replaced by the correspondent Unicode space symbol in Table 2.

Algorithm 1 WATERMARK EMBEDDING

```
/* Text string of n Unicode characters c */
T = {c_1, ..., c_n}
/* List of confusable original symbols */
Originals = {U+002c,U+002d,U+002e,...}
/* List of confusable duplicate symbols */
Duplicates = {U+a4f9,U+2010,U+a4f8,...}
/* List of confusable white spaces */
Spaces = {U+0020,U+2002,U+2005,...}
/* List of all confusables*/
Confusables = Originals ∪ Spaces
W = [b_1, ..., b_64] /* Watermark bit array */
GetDuplicate : Originals → Duplicates
GetSpace : {000, ..., 111} → Spaces
WT = [] /* Watermarked text */
for all c ∈ T do
    if c ∈ Confusables ∧ |W| ≠ 0 then
        if c ∈ Spaces then
            bits = Pop(W, 3)
            lshift(bits, 3 − len(bits))
            c = GetSpace(bits)
        else
            bit = Pop(W, 1)
            if bit=1 then
                c = GetDuplicate(c)
            end if
        end if
    end if
    Append(WT, c)
end for
return WT
```

The embedding Algorithm 1 cost is constant in the best case, depending only on the number of bits in the hash, while linear on the number of characters in the text in the worst case scenario. Using Algorithm 1 it is possible to embed a watermark in a very short text, up to a minimum boundary. In order to determine a theoretical minimum boundary on text length, we consider the most extreme case in which our text is made of white spaces only. In this case the needed characters would be only 22, since we can embed 3 bits for each white space. In less extreme but still unusual scenario, we can consider a single white space between words of one character, no double white spaces and no white space at the end of the text. In these conditions, the minimum lenght to embed a 64bit payload would be only 33 characters. These are definitely extreme scenarios which could hardly occur in real documents or messages. After extensively experiment with more than 1.8 million New York Times articles [22], the minimum number of characters needed to encode the watermark has been of only 46 characters (white space included), while on average the needed characters are less than 101 (white space included). We summarize the experiment results in Table 3 with the boundaries and averages of length, also when longer hash functions are used instead of the 64bit SipHash. We refer to Section 4 for more detailed results on the evaluation.

	SipHash (64bit)	MD5 (128bit)	SHA-1 (160bit)	SHA-2 (224bit)
Spaces	22	43	54	75
1-char words	33	65	81	113
NYT MIN	46	93	116	163
NYT AVG	101.3	197.7	246.5	344

Table 3: Minimum text length for length-preserving watermarking on SipHash and other well known cryptographic hash functions.

3.4 Unicode watermark extraction

As we have already said, there are two types of watermarking: blind and non-blind. In blind watermarking the watermark can be extracted without the original text, while in non-blind watermarking the original text is needed to extract the watermark. Because the original text is supposed to use standard Unicode latin letters, while our watermarked text contains also similar but non standard Unicode symbols, the watermark can be extracted from the watermarked text using the reverse mapping suggested by Table 1. Our approach falls therefore under the blind watermarking category. Another consequence of the embedding algorithm is that the watermark is invisible and not readable, but it is detectable, since it is sufficient to examine the code representation of each symbol to determine whether it is standard or not.
The extraction algorithm (Algorithm 2) is therefore rather straightforward. For each confusable symbol found (in Table 1 or Table 2), if the symbol is in standard format then the hash bit is "0" (or "000" for spaces), otherwise it is a "1" (or one of the 7 non-zero bits in Table 2 for white spaces). The hash bits are extracted in this way from the most significant to the least significant.
As for the embedding algorithm, the extraction algorithm

cost is constant in the best case, depending only on the number of bits in the hash, while linear on the number of characters in the text in the worst case scenario.

Algorithm 2 WATERMARK EXTRACTION

```
/* Text string of n Unicode characters c */
T = {c_1, ..., c_n}
/* List of confusable original symbols */
Originals = {U+002c,U+002d,U+002e,...}
/* List of confusable duplicate symbols */
Duplicates = {U+a4f9,U+2010,U+a4f8,...}
/* List of confusable white spaces */
Spaces = {U+0020,U+2002,U+2005,...}
W = [] /* Watermark bit array */
SpaceMap : Spaces → {000, ..., 111} /* Reverse mapping function*/
for all c in T do
    if c ∈ Spaces then
        Append(W, SpaceMap(c))
    else if c ∈ Originals then
        Append(W, 0)
    else if c ∈ Duplicates then
        Append(W, 1)
    end if
    if |W| ≥ 64 then
        break
    end if
end for
return W[: 64]
```

3.5 Authorship verification

The authorship verification process allows the original owner who generated the watermark to prove its authorship. The process requires to be robust, that is no attacker (with limited computational resources) should be able to success in the process. In non-blind watermarking this robustness is provided by the impossibility to extract the watermark. In our blind approach, in which an attacker can extract the watermark knowing our Unicode mapping, robustness is obtained through the password and the hash function. The authorship verification therefore depends on the reproducibility of the watermark, that is possible for the key owner only.

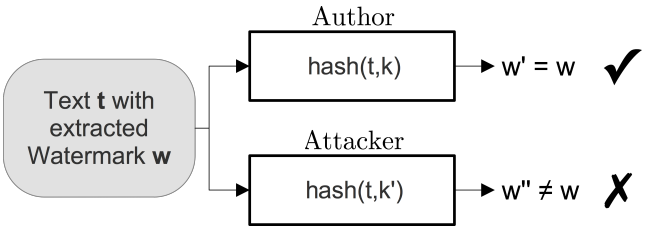


Figure 3: The password k is a proof of authorship. Once the watermark is extracted, only the author with the original password k is able to reproduce it.

More specifically, robustness is ensured by the one-way function of the SipHash MAC. In fact, it is impossible to reproduce the watermark without having the password, unless the attacker try exhaustively all the 16-bytes keys. In Figure 3 this process is described in 3 steps:

1. Given a watermarked text, both the author and the attacker can extract the original text t and the watermark w using the mapping table.

2. The author of the watermark apply the SipHash function to the text t using his password k, thus obtaining a watermark w'. The attacker apply the SipHash function on t as well, however using a wrong password k', obtaining a watermark w''.

3. The reproduced watermarks w' and w'' are compared to the original watermark w, proving the authenticity of the key k and as a result the authorship of its owner.

While the chosen MAC is considered to be secure in most contexts, given enough time or computational power for the attacker or new developments in cryptographic algorithms could raise the hash function requirements. For this reason we show in Table 3 alternative hash functions and the related boundaries and averages for the text length requirement.

4. EVALUATION

We evaluate the two most important properties of the presented approach: the small text length requirements for embedding the watermark and the imperceptibility of the symbols substitutions.

In order to investigate the text length requirements in real world text samples, we take in consideration the New York Times Corpus [22], a collection of 1.8 million articles from the New York Times newspaper spanning from 1987 to 2007. More specifically, to stress the fact that this watermark approach can be successfully applied to small texts, we extracted the lead paragraph[3] from all articles and experimented on these portions of text.
The number of characters required to embed a full payload (e.g. a 64bit watermark) strictly depends on how many confusable symbols are among those characters. More confusable symbols in a text means less characters needed to embed the payload (especially if the confusables are white space, accounting for more embedding space).
We tested the embedding algorithm using cryptographic hash functions of different hash length, from 64 bits (SipHash) to 224 bits (SHA-2), in order to statistically find out the probability of successfully embed a watermark for each specific text length.
In Figure 4 we show that using our approach it is possible to embed a SipHash watermark in the 94.51% of the leading paragraphs. Of the remaining 5.49%, 3.94% are too short paragraphs while the 1.55% are paragraphs without enough confusables to embed the payload. Considering the other hash sizes, the percentage of successfully watermarked articles drop to 91.83% using a 128bit hash, 74.38% using a 160bit hash and 63.97% with a 224bit hash.
Apart from the percentage of watermarkable paragraphs, the plot in Figure 4 gives valuable information on the odds of successfully embed the watermark within the first n characters. In other words, this is the statistical probability of successfully watermark a text of n characters for each hash size.

[3]The *lead paragraph* field of the article as defined by New York Times.

For instance, to get a chance greater than 50% of embedding a watermark, the text must have at least 100 characters using SipHash (64bit), 199 characters using MD5 (128bit), 253 characters with SHA-1 (160bit) and 357 characters with SHA-2 (224bit).

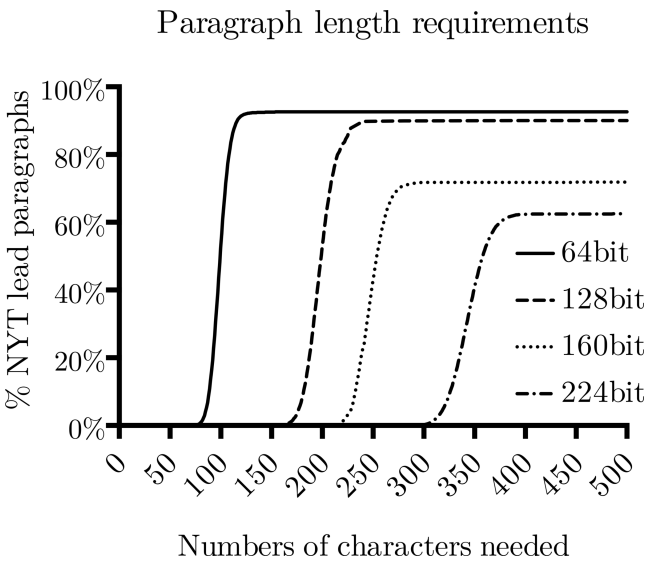


Figure 4: Percentage of leading paragraphs in NYT Corpus that can be watermarked using only the first n characters. Results are shown for different hash lengths.

In the process of watermark embedding some of the original text characters are replaced with other symbols. In order for the watermark to be imperceptible, it is crucial to show the watermarked text is indistinguishable from the original text As shown in Figure 5, the differences between the original and the watermarked paragraph are hardly noticeable by the human eye. In fact, the most perceptible difference regards the horizontal spacing: using 8 different white spaces of different sizes slightly affects the words position and consequently the length of the paragraph.

Tina Marie Soha letter on June 27 article scores court ruling banning Pledge of Allegiance in schools;	A
Tina Marie Soha letter on June 27 article scores court ruling banning Pledge of Allegiance in schools;	B

Figure 5: A lead paragraph from a New York Time article before watermarking (A) and after watermarking (B).

In order to check and show the degree imperceptibility of each homoglyph in different fonts, we show in Table 4 all the confusable symbols for 3 different well-known fonts in their original encoding (left columns) and their duplicate encoding (right columns). All these sans-serifs fonts are used in the most known web platforms, comprising Facebook, Twitter and Google. Table 4 show little to no noticeable differences

	Arial		Helvetica		Helv. Neue	
Symb.	**Or.**	**Dup.**	**Or.**	**Dup.**	**Or.**	**Dup.**
c	c	c	c	c	c	c
C	C	C	C	C	C	C
d	d	d	d	d	d	d
D	D	D	D	D	D	D
i	i	i	i	i	i	i
l	l	l	l	l	l	l
L	L	L	L	L	L	L
M	M	M	M	M	M	M
v	v	v	v	v	v	v
V	V	V	V	V	V	V
x	x	x	x	x	x	x
X	X	X	X	X	X	X
j	j	j	j	j	j	j
K	K	K	K	K	K	K
;	;	;	;	;	;	;
-	-	-	-	-	-	-

Table 4: Confusable symbols rendering (original and duplicate) for the three most known web fonts.

in letter rendering when the duplicate symbol is used instead of the original one.

5. CONCLUSIONS AND FUTURE WORKS

Copyright protection and authentication about digital contents has become a challenging research problem, especially due to the widespread use of internet and the increasingly diffusion of sharing platforms.

In this paper, we have presented a text watermarking method able to embed a password based watermark in latin-based alphabet texts. The method blends the original text and a user password through a hash function in order to compute the watermark. Then, the algorithm exploits the homoglyph Unicode characters to embed the watermark in the original text. The proposed method does not modify neither the syntactic structure nor the semantic form of the original text. The watermarked text preserves the original length and appears visually indistinguishable. Moreover, we demonstrated that the shortest length in order to apply a 64bit watermark is of only 46 characters. The latter is an important feature that makes possible to use the method in several other contexts, like: microblogging, short message communications and social network posts.

As a future work, we plan to stress out our method on some real cases. In particular, we want to extensively evaluate the watermarking embedding and extraction process on different social networks. That means address issues concerning URLs and special characters coding that currently must be skipped in the embedding process. Moreover, social networks have some automatic cleaning procedures that can erase the watermark from the text of the user-submitted post. Future works also include evaluating the distribution chain of a given text on different platforms, by exploiting the method's capability to be applied multiple times using different passwords.

6. ACKNOWLEDGMENTS

This work has been supported in part by the Italian Ministry of Education, Universities and Research, OPLON project (SCN_00176).

7. REFERENCES

[1] T. Amano and D. Misaki. A feature calibration method for watermarking of document images. In *Proceedings of the Fifth International Conference on Document Analysis and Recognition*, pages 91–94. IEEE, 1999.

[2] M. J. Atallah, V. Raskin, M. Crogan, C. Hempelmann, F. Kerschbaum, D. Mohamed, and S. Naik. Natural language watermarking: Design, analysis, and a proof-of-concept implementation. In *Information Hiding*, pages 185–200. Springer, 2001.

[3] J.-P. Aumasson and D. J. Bernstein. Siphash: a fast short-input prf. In *Progress in Cryptology-INDOCRYPT 2012*, pages 489–508. Springer, 2012.

[4] A. K. Bhattacharjya and H. Ancin. Data embedding in text for a copier system. In *Image Processing, 1999. ICIP 99. Proceedings. 1999 International Conference on*, volume 2, pages 245–249. IEEE, 1999.

[5] J. T. Brassil, S. Low, N. F. Maxemchuk, and L. O. Gorman. Electronic marking and identification techniques to discourage document copying. *Selected Areas in Communications, IEEE Journal on*, 13(8):1495–1504, 1995.

[6] M. Davis and M. Suignard. Unicode security mechanisms. Unicode technical standard #39, Unicode. http://www.unicode.org/reports/tr39/.

[7] D. Gross-Amblard. Query-preserving watermarking of relational databases and xml documents. In *Proceedings of the twenty-second ACM SIGMOD-SIGACT-SIGART symposium on Principles of database systems*, pages 191–201. ACM, 2003.

[8] F. Hartung and M. Kutter. Multimedia watermarking techniques. *Proceedings of the IEEE*, 87(7):1079–1107, 1999.

[9] S. Hosmani, H. R. Bhat, and K. Chandrasekaran. Dual stage text steganography using unicode homoglyphs. In *Security in Computing and Communications*, pages 265–276. Springer, 2015.

[10] D. Huang and H. Yan. Interword distance changes represented by sine waves for watermarking text images. *Circuits and Systems for Video Technology, IEEE Transactions on*, 11(12):1237–1245, 2001.

[11] Z. Jalil and A. M. Mirza. A review of digital watermarking techniques for text documents. In *Information and Multimedia Technology, 2009. ICIMT'09. International Conference on*, pages 230–234. IEEE, 2009.

[12] S. Katzenbeisser and F. Petitcolas. *Information hiding techniques for steganography and digital watermarking*. Artech house, 2000.

[13] M. Kaur and K. Mahajan. An existential review on text watermarking techniques. *International Journal of Computer Applications*, 120(18), 2015.

[14] Y.-W. Kim, K.-A. Moon, and I.-S. Oh. A text watermarking algorithm based on word classification and inter-word space statistics. In *Proceedings of the Seventh International Conference on Document Analysis and Recognition*, pages 775–779. IEEE, 2003.

[15] Y.-W. Kim and I.-S. Oh. Watermarking text document images using edge direction histograms. *Pattern Recognition Letters*, 25(11):1243–1251, 2004.

[16] J. C. Lai and C. B. Graber. Is digital text-watermarking the long-desired user friendly digital rights management? copyright and fundamental values from a comparative perspective. *European Intellectual Property Review, published by Sweet & Maxwell*, 2016.

[17] S. H. Low, N. F. Maxemchuk, and A. M. Lapone. Document identification for copyright protection using centroid detection. *Communications, IEEE Transactions on*, 46(3):372–383, 1998.

[18] H. M. Meral, B. Sankur, A. S. Özsoy, T. Güngör, and E. Sevinç. Natural language watermarking via morphosyntactic alterations. *Computer Speech & Language*, 23(1):107–125, 2009.

[19] N. Mir. Copyright for web content using invisible text watermarking. *Computers in Human Behavior*, 30:648–653, 2014.

[20] R. Patel and P. Bhatt. A review paper on digital watermarking and its techniques. *International Journal of Computer Applications*, 110(1):10–13, 2015.

[21] L. Y. Por, K. Wong, and K. O. Chee. Unispach: A text-based data hiding method using unicode space characters. *Journal of Systems and Software*, 85(5):1075–1082, 2012.

[22] E. Sandhaus. The new york times annotated corpus. *Linguistic Data Consortium, Philadelphia*, 6(12):e26752, 2008.

[23] M. Topkara, C. M. Taskiran, and E. J. Delp III. Natural language watermarking. In *Electronic Imaging 2005*, pages 441–452. International Society for Optics and Photonics, 2005.

[24] U. Topkara, M. Topkara, and M. J. Atallah. The hiding virtues of ambiguity: quantifiably resilient watermarking of natural language text through synonym substitutions. In *Proceedings of the 8th workshop on Multimedia and security*, pages 164–174. ACM, 2006.

[25] X. Zhou, W. Zhao, Z. Wang, and L. Pan. Security theory and attack analysis for text watermarking. In *E-Business and Information System Security, 2009. EBISS'09. International Conference on*, pages 1–6. IEEE, 2009.